

MATRIX BASED KEYPAD DTMF TONE GENERATOR

This is divided into 3 sections audio.v topmodule.v and buttons.v

AUDIO.V

```
module audio(
    input clk,
    input [4:0] key,
    output reg audio_l, audio_r

);

// reg [4:0] key;
// always @(*) begin
//     key <= 5'd1;
// end

parameter clkfreq=12000000;

reg [15:0] freq_r ;
reg [15:0] freq_c ;
wire key_pressed = (key >= 5'd1 && key <= 5'd16);
wire [31:0] phase_inc_r;
wire [31:0] phase_inc_c;
assign phase_inc_r = freq_r * 358; //freq*2**32/clock frequency
assign phase_inc_c = freq_c * 358;

always @(*) begin
    case (key)
        5'd1,5'd2,5'd3 : freq_r = 697;
        5'd4,5'd5,5'd6 : freq_r = 770;
        5'd7,5'd8,5'd9 : freq_r = 852;

        default: freq_r = 0;
    endcase
end

always @(*) begin
    case (key)
        5'd1,5'd4,5'd7 : freq_c = 1209;
        5'd2,5'd5,5'd8 : freq_c = 1336;
        5'd3,5'd6,5'd9 : freq_c = 1477;

        default: freq_c = 0;
    endcase
end

reg signed [15:0] sine_out_r;
reg [31:0] counter_r;
wire [7:0] addr_r;
```

```

reg signed [15:0] sine_out_c;
reg [31:0] counter_c;
wire [7:0] addr_c;

assign addr_r = counter_r[31:24];
assign addr_c = counter_c[31:24];
always @(posedge clk) begin
    if (key_pressed) begin
        counter_r <= counter_r + phase_inc_r;
        counter_c <= counter_c + phase_inc_c;
    end else begin
        counter_r <= 32'd0;
        counter_c <= 32'd0;
    end
end

wire [15:0] sample_r= sine_out_r + 16'd32768;
wire [15:0] sample_c= sine_out_c + 16'd32768;
wire [16:0] mixer= sample_r + sample_c;

reg [17:0] acc = 0;
always @(posedge clk) begin
    if (key_pressed) begin

acc <= {1'b0, acc[16:0]} + mixer;
        end else begin

            acc <= 18'd0;
        end
end
always @(posedge clk) begin
    if(key_pressed) begin
        audio_l <= acc[17];
        audio_r <= acc[17];
    end else begin
        audio_l <= 1'b0;
        audio_r <= 1'b0;
    end
end

end

sinout sinout_r (
    .addr(addr_r),
    .sine_out(sine_out_r)
);
sinout sinout_c (
    .addr(addr_c),
    .sine_out(sine_out_c)
);

```

```
endmodule
```

```
BUTTONS.V
```

```
module buttons (
```

```
    input clk,
```

```
    input r1, r2, r3,
```

```
    output reg c1, c2, c3,
```

```
    output reg [4:0] key,
```

```
    // output reg led_blue_n,
```

```
    // output reg led_amber_n,
```

```
    // output reg led_rgb_green_n
```

```
);
```

```
    parameter STATE_SCAN_C1= 4'b0000, STATE_READ_C1= 4'b0001,  
    STATE_READ_C1_WAIT = 4'b0010;
```

```
    parameter STATE_SCAN_C2= 4'b0011, STATE_READ_C2= 4'b0100,  
    STATE_READ_C2_WAIT = 4'b0101;
```

```
    parameter STATE_SCAN_C3= 4'b0110, STATE_READ_C3= 4'b0111,  
    STATE_READ_C3_WAIT = 4'b1000;
```

```
    reg [3:0] curr_state = STATE_SCAN_C1;
```

```
    reg [3:0] next_state;
```

```
    reg [2:0] ff1, ff2;
```

```
    reg [8:0] det_key = 9'b000000000;
```

```
    reg [22:0] clk_div = 0;
```

```
    always @(posedge clk) begin
```

```
        clk_div <= clk_div + 1;
```

```
    end
```

```
    wire scan_clk = clk_div[15];
```

```
    wire [2:0] row = {r3, r2, r1};
```

```
    always @(posedge scan_clk) begin
```

```
        ff1 <= row;
```

```
        ff2 <= ff1;
```

```
        curr_state <= next_state;
```

```
    end
```

```
    always @(posedge scan_clk) begin
```

```
        case (curr_state)
```

```
            STATE_READ_C1_WAIT: begin
```

```
                det_key[0] <= ff2[0];
```

```
                det_key[3] <= ff2[1];
```

```
                det_key[6] <= ff2[2];
```

```
            end
```

```
            STATE_READ_C2_WAIT: begin
```

```
                det_key[1] <= ff2[0];
```

```
                det_key[4] <= ff2[1];
```

```

        det_key[7] <= ff2[2];
    end
    STATE_READ_C3_WAIT: begin
        det_key[2] <= ff2[0];
        det_key[5] <= ff2[1];
        det_key[8] <= ff2[2];
    end
endcase
end

always @(*) begin
    next_state = curr_state;
    c1 = 1'b0; c2 = 1'b0; c3 = 1'b0;

    case(curr_state)
        STATE_SCAN_C1: begin
            c1 = 1'b1;
            next_state = STATE_READ_C1;
        end
        STATE_READ_C1: begin
            c1 = 1'b1;
            next_state = STATE_READ_C1_WAIT;
        end
        STATE_READ_C1_WAIT: begin
            c1 = 1'b1;
            next_state = STATE_SCAN_C2;
        end

        STATE_SCAN_C2: begin
            c2 = 1'b1;
            next_state = STATE_READ_C2;
        end
        STATE_READ_C2: begin
            c2 = 1'b1;
            next_state = STATE_READ_C2_WAIT;
        end
        STATE_READ_C2_WAIT: begin
            c2 = 1'b1;
            next_state = STATE_SCAN_C3;
        end

        STATE_SCAN_C3: begin
            c3 = 1'b1;
            next_state = STATE_READ_C3;
        end
        STATE_READ_C3: begin
            c3 = 1'b1;
            next_state = STATE_READ_C3_WAIT;
        end
        STATE_READ_C3_WAIT: begin
            c3 = 1'b1;

```

```

        next_state = STATE_SCAN_C1;
    end

    default: next_state = STATE_SCAN_C1;
endcase
end

always @(*) begin
    case (det_key)

        9'b000000001: begin
            key= 5'd1;
        end
        9'b000000010: begin
            key= 5'd2;
        end
        9'b000000100: begin
            key= 5'd3;
        end

        9'b000001000: begin
            key= 5'd4;
        end
        9'b000010000: begin
            key= 5'd5;
        end
        9'b000100000: begin
            key= 5'd6;
        end

        9'b001000000: begin
            key= 5'd7;
        end
        9'b010000000: begin
            key= 5'd8;
        end
        9'b100000000: begin
            key= 5'd9;
        end

        default: begin
            key= 5'd0;
        end
    endcase
end

endmodule

```

TOPMODULE.V

```
module top_module(
    input clk,
    output audio_l,audio_r,
        input r1,r2,
        input r3,r4,
        output c1,c2,
        output c3,
        output c4,
    //      output reg led_blue_n,
    // output reg led_amber_n,
    // output reg led_rgb_green_n
);

wire [4:0] key;
wire audio_l,audio_r;

audio audio_inst(
    .clk(clk),
    .key(key),
    .audio_l(audio_l),
    .audio_r(audio_r)
);
buttons button_inst5(
    .clk(clk),
    .r1(r1),
    .r2(r2),
    .r3(r3),
    .c1(c1),
    .c2(c2),
    .c3(c3),
    .key(key),
    // .led_blue_n(led_blue_n),
    // .led_amber_n(led_amber_n),
    // .led_rgb_green_n(led_rgb_green_n)
);
```

SINOUT.v

```
module sinout(
    input [7:0] addr,
    output reg signed [15:0] sine_out
);
```

```
always @(*) begin
```

```
    case(addr)
        8'd0  : sine_out = 16'sd0;
        8'd1  : sine_out = 16'sd804;
        8'd2  : sine_out = 16'sd1608;
```

8'd3 : sine_out = 16'sd2410;
8'd4 : sine_out = 16'sd3212;
8'd5 : sine_out = 16'sd4011;
8'd6 : sine_out = 16'sd4808;
8'd7 : sine_out = 16'sd5602;
8'd8 : sine_out = 16'sd6393;
8'd9 : sine_out = 16'sd7179;
8'd10 : sine_out = 16'sd7962;
8'd11 : sine_out = 16'sd8739;
8'd12 : sine_out = 16'sd9512;
8'd13 : sine_out = 16'sd10278;
8'd14 : sine_out = 16'sd11039;
8'd15 : sine_out = 16'sd11793;
8'd16 : sine_out = 16'sd12539;
8'd17 : sine_out = 16'sd13279;
8'd18 : sine_out = 16'sd14010;
8'd19 : sine_out = 16'sd14732;
8'd20 : sine_out = 16'sd15446;
8'd21 : sine_out = 16'sd16151;
8'd22 : sine_out = 16'sd16846;
8'd23 : sine_out = 16'sd17530;
8'd24 : sine_out = 16'sd18204;
8'd25 : sine_out = 16'sd18868;
8'd26 : sine_out = 16'sd19519;
8'd27 : sine_out = 16'sd20159;
8'd28 : sine_out = 16'sd20787;
8'd29 : sine_out = 16'sd21403;
8'd30 : sine_out = 16'sd22005;
8'd31 : sine_out = 16'sd22594;
8'd32 : sine_out = 16'sd23170;
8'd33 : sine_out = 16'sd23731;
8'd34 : sine_out = 16'sd24279;
8'd35 : sine_out = 16'sd24811;
8'd36 : sine_out = 16'sd25329;
8'd37 : sine_out = 16'sd25832;
8'd38 : sine_out = 16'sd26319;
8'd39 : sine_out = 16'sd26790;
8'd40 : sine_out = 16'sd27245;
8'd41 : sine_out = 16'sd27683;
8'd42 : sine_out = 16'sd28105;
8'd43 : sine_out = 16'sd28510;
8'd44 : sine_out = 16'sd28898;
8'd45 : sine_out = 16'sd29268;
8'd46 : sine_out = 16'sd29621;
8'd47 : sine_out = 16'sd29956;
8'd48 : sine_out = 16'sd30273;
8'd49 : sine_out = 16'sd30571;
8'd50 : sine_out = 16'sd30852;
8'd51 : sine_out = 16'sd31113;
8'd52 : sine_out = 16'sd31356;
8'd53 : sine_out = 16'sd31580;
8'd54 : sine_out = 16'sd31785;

8'd55 : sine_out = 16'sd31971;
8'd56 : sine_out = 16'sd32137;
8'd57 : sine_out = 16'sd32285;
8'd58 : sine_out = 16'sd32412;
8'd59 : sine_out = 16'sd32521;
8'd60 : sine_out = 16'sd32609;
8'd61 : sine_out = 16'sd32678;
8'd62 : sine_out = 16'sd32728;
8'd63 : sine_out = 16'sd32757;
8'd64 : sine_out = 16'sd32767;
8'd65 : sine_out = 16'sd32757;
8'd66 : sine_out = 16'sd32728;
8'd67 : sine_out = 16'sd32678;
8'd68 : sine_out = 16'sd32609;
8'd69 : sine_out = 16'sd32521;
8'd70 : sine_out = 16'sd32412;
8'd71 : sine_out = 16'sd32285;
8'd72 : sine_out = 16'sd32137;
8'd73 : sine_out = 16'sd31971;
8'd74 : sine_out = 16'sd31785;
8'd75 : sine_out = 16'sd31580;
8'd76 : sine_out = 16'sd31356;
8'd77 : sine_out = 16'sd31113;
8'd78 : sine_out = 16'sd30852;
8'd79 : sine_out = 16'sd30571;
8'd80 : sine_out = 16'sd30273;
8'd81 : sine_out = 16'sd29956;
8'd82 : sine_out = 16'sd29621;
8'd83 : sine_out = 16'sd29268;
8'd84 : sine_out = 16'sd28898;
8'd85 : sine_out = 16'sd28510;
8'd86 : sine_out = 16'sd28105;
8'd87 : sine_out = 16'sd27683;
8'd88 : sine_out = 16'sd27245;
8'd89 : sine_out = 16'sd26790;
8'd90 : sine_out = 16'sd26319;
8'd91 : sine_out = 16'sd25832;
8'd92 : sine_out = 16'sd25329;
8'd93 : sine_out = 16'sd24811;
8'd94 : sine_out = 16'sd24279;
8'd95 : sine_out = 16'sd23731;
8'd96 : sine_out = 16'sd23170;
8'd97 : sine_out = 16'sd22594;
8'd98 : sine_out = 16'sd22005;
8'd99 : sine_out = 16'sd21403;
8'd100 : sine_out = 16'sd20787;
8'd101 : sine_out = 16'sd20159;
8'd102 : sine_out = 16'sd19519;
8'd103 : sine_out = 16'sd18868;
8'd104 : sine_out = 16'sd18204;
8'd105 : sine_out = 16'sd17530;
8'd106 : sine_out = 16'sd16846;

8'd107 : sine_out = 16'sd16151;
8'd108 : sine_out = 16'sd15446;
8'd109 : sine_out = 16'sd14732;
8'd110 : sine_out = 16'sd14010;
8'd111 : sine_out = 16'sd13279;
8'd112 : sine_out = 16'sd12539;
8'd113 : sine_out = 16'sd11793;
8'd114 : sine_out = 16'sd11039;
8'd115 : sine_out = 16'sd10278;
8'd116 : sine_out = 16'sd9512;
8'd117 : sine_out = 16'sd8739;
8'd118 : sine_out = 16'sd7962;
8'd119 : sine_out = 16'sd7179;
8'd120 : sine_out = 16'sd6393;
8'd121 : sine_out = 16'sd5602;
8'd122 : sine_out = 16'sd4808;
8'd123 : sine_out = 16'sd4011;
8'd124 : sine_out = 16'sd3212;
8'd125 : sine_out = 16'sd2410;
8'd126 : sine_out = 16'sd1608;
8'd127 : sine_out = 16'sd804;
8'd128 : sine_out = 16'sd0;
8'd129 : sine_out = -16'sd804;
8'd130 : sine_out = -16'sd1608;
8'd131 : sine_out = -16'sd2410;
8'd132 : sine_out = -16'sd3212;
8'd133 : sine_out = -16'sd4011;
8'd134 : sine_out = -16'sd4808;
8'd135 : sine_out = -16'sd5602;
8'd136 : sine_out = -16'sd6393;
8'd137 : sine_out = -16'sd7179;
8'd138 : sine_out = -16'sd7962;
8'd139 : sine_out = -16'sd8739;
8'd140 : sine_out = -16'sd9512;
8'd141 : sine_out = -16'sd10278;
8'd142 : sine_out = -16'sd11039;
8'd143 : sine_out = -16'sd11793;
8'd144 : sine_out = -16'sd12539;
8'd145 : sine_out = -16'sd13279;
8'd146 : sine_out = -16'sd14010;
8'd147 : sine_out = -16'sd14732;
8'd148 : sine_out = -16'sd15446;
8'd149 : sine_out = -16'sd16151;
8'd150 : sine_out = -16'sd16846;
8'd151 : sine_out = -16'sd17530;
8'd152 : sine_out = -16'sd18204;
8'd153 : sine_out = -16'sd18868;
8'd154 : sine_out = -16'sd19519;
8'd155 : sine_out = -16'sd20159;
8'd156 : sine_out = -16'sd20787;
8'd157 : sine_out = -16'sd21403;
8'd158 : sine_out = -16'sd22005;

8'd159 : sine_out = -16'sd22594;
8'd160 : sine_out = -16'sd23170;
8'd161 : sine_out = -16'sd23731;
8'd162 : sine_out = -16'sd24279;
8'd163 : sine_out = -16'sd24811;
8'd164 : sine_out = -16'sd25329;
8'd165 : sine_out = -16'sd25832;
8'd166 : sine_out = -16'sd26319;
8'd167 : sine_out = -16'sd26790;
8'd168 : sine_out = -16'sd27245;
8'd169 : sine_out = -16'sd27683;
8'd170 : sine_out = -16'sd28105;
8'd171 : sine_out = -16'sd28510;
8'd172 : sine_out = -16'sd28898;
8'd173 : sine_out = -16'sd29268;
8'd174 : sine_out = -16'sd29621;
8'd175 : sine_out = -16'sd29956;
8'd176 : sine_out = -16'sd30273;
8'd177 : sine_out = -16'sd30571;
8'd178 : sine_out = -16'sd30852;
8'd179 : sine_out = -16'sd31113;
8'd180 : sine_out = -16'sd31356;
8'd181 : sine_out = -16'sd31580;
8'd182 : sine_out = -16'sd31785;
8'd183 : sine_out = -16'sd31971;
8'd184 : sine_out = -16'sd32137;
8'd185 : sine_out = -16'sd32285;
8'd186 : sine_out = -16'sd32412;
8'd187 : sine_out = -16'sd32521;
8'd188 : sine_out = -16'sd32609;
8'd189 : sine_out = -16'sd32678;
8'd190 : sine_out = -16'sd32728;
8'd191 : sine_out = -16'sd32757;
8'd192 : sine_out = -16'sd32767;
8'd193 : sine_out = -16'sd32757;
8'd194 : sine_out = -16'sd32728;
8'd195 : sine_out = -16'sd32678;
8'd196 : sine_out = -16'sd32609;
8'd197 : sine_out = -16'sd32521;
8'd198 : sine_out = -16'sd32412;
8'd199 : sine_out = -16'sd32285;
8'd200 : sine_out = -16'sd32137;
8'd201 : sine_out = -16'sd31971;
8'd202 : sine_out = -16'sd31785;
8'd203 : sine_out = -16'sd31580;
8'd204 : sine_out = -16'sd31356;
8'd205 : sine_out = -16'sd31113;
8'd206 : sine_out = -16'sd30852;
8'd207 : sine_out = -16'sd30571;
8'd208 : sine_out = -16'sd30273;
8'd209 : sine_out = -16'sd29956;
8'd210 : sine_out = -16'sd29621;

```
8'd211 : sine_out = -16'sd29268;
8'd212 : sine_out = -16'sd28898;
8'd213 : sine_out = -16'sd28510;
8'd214 : sine_out = -16'sd28105;
8'd215 : sine_out = -16'sd27683;
8'd216 : sine_out = -16'sd27245;
8'd217 : sine_out = -16'sd26790;
8'd218 : sine_out = -16'sd26319;
8'd219 : sine_out = -16'sd25832;
8'd220 : sine_out = -16'sd25329;
8'd221 : sine_out = -16'sd24811;
8'd222 : sine_out = -16'sd24279;
8'd223 : sine_out = -16'sd23731;
8'd224 : sine_out = -16'sd23170;
8'd225 : sine_out = -16'sd22594;
8'd226 : sine_out = -16'sd22005;
8'd227 : sine_out = -16'sd21403;
8'd228 : sine_out = -16'sd20787;
8'd229 : sine_out = -16'sd20159;
8'd230 : sine_out = -16'sd19519;
8'd231 : sine_out = -16'sd18868;
8'd232 : sine_out = -16'sd18204;
8'd233 : sine_out = -16'sd17530;
8'd234 : sine_out = -16'sd16846;
8'd235 : sine_out = -16'sd16151;
8'd236 : sine_out = -16'sd15446;
8'd237 : sine_out = -16'sd14732;
8'd238 : sine_out = -16'sd14010;
8'd239 : sine_out = -16'sd13279;
8'd240 : sine_out = -16'sd12539;
8'd241 : sine_out = -16'sd11793;
8'd242 : sine_out = -16'sd11039;
8'd243 : sine_out = -16'sd10278;
8'd244 : sine_out = -16'sd9512;
8'd245 : sine_out = -16'sd8739;
8'd246 : sine_out = -16'sd7962;
8'd247 : sine_out = -16'sd7179;
8'd248 : sine_out = -16'sd6393;
8'd249 : sine_out = -16'sd5602;
8'd250 : sine_out = -16'sd4808;
8'd251 : sine_out = -16'sd4011;
8'd252 : sine_out = -16'sd3212;
8'd253 : sine_out = -16'sd2410;
8'd254 : sine_out = -16'sd1608;
8'd255 : sine_out = -16'sd804;
    default: sine_out = 16'sd0;
```

```
endcase
```

```
end
```

endmodule